

Architecture Design Report

This document presents the architectural design of the system, including its context, logical structure, runtime behavior, and deployment details.

1. System Requirements

System Name: ResuFit: Precision CV Matching System

Architecture Style: Microservices

Functional Requirements

- Submit CV
- View and Edit CV
- Answer Screening Questions
- Get Rating for CV
- Parse CV
- Compare CV with Job Requirements
- Validate and Score Applicant Responses
- Generate Percentage-Based Result

Non-Functional Requirements

- Performance

Description: The system **MUST** respond to user requests within a reasonable time frame.

- Reliability

Description: The system **MUST** be able to recover from failures and maintain data consistency.

- Security

Description: The system **MUST** protect user data and prevent unauthorized access.

- Privacy

Description: The system **MUST** ensure that user data is handled in accordance with applicable laws and regulations.

- Scalability

Description: The system **MUST** be able to handle a large number of users and requests without compromising performance.

- Usability

Description: The system **MUST** provide an intuitive and user-friendly interface that is easy to navigate.

- Maintainability

Description: The system MUST be designed to be easily maintained and updated.

- Compatibility

Description: The system MUST be compatible with a variety of devices and browsers.

- Integration

Description: The system MUST be able to integrate with other systems and services.

2. Formal Architecture Specification (ACME ADL)

The following section provides a formal architectural specification using ACME ADL, capturing components, con-

```
System ResuFit:PrecisionCVMatchingSystem {
  Property view : String = "runtime";
  Property architectureStyle : String = "Microservices";
  Property source : String = "AI";
  Component CVParser = new Component {
    Port in;
    Port out;
    Property responsibility : String = "['Parse CV', 'Extract key information from CV', 'Support for"];
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component ScreeningQuestionEngine = new Component {
    Port in;
    Port out;
    Property responsibility : String = "['Answer Screening Questions', 'Validate and Score Applicant"];
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component JobMatchingAlgorithm = new Component {
    Port in;
    Port out;
    Property responsibility : String = "['Compare CV with Job Requirements', 'Calculate cosine simil"];
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component UserInterface = new Component {
    Port in;
    Port out;
    Property responsibility : String = "['Submit CV', 'View and Edit CV', 'Present screening questio"];
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component RatingEngine = new Component {
    Port in;
    Port out;
    Property responsibility : String = "['Get Rating for CV', 'Calculate rating based on CV and job"];
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
}
```

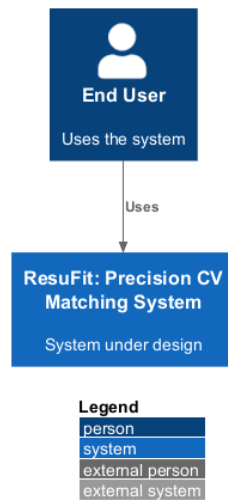
```

}
Component ChatGPTIntegration = new Component {
    Port in;
    Port out;
    Property responsibility : String = "['Validate and Score Applicant Responses', 'Compare applicant responses']";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Attachment CVParser.out to JobMatchingAlgorithm.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment CVParser.out to RatingEngine.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment CVParser.out to UserInterface.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = false;
    Property backpressure : String = "bounded";
}
Attachment ScreeningQuestionEngine.out to UserInterface.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = false;
    Property backpressure : String = "bounded";
}
Attachment JobMatchingAlgorithm.out to UserInterface.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = false;
    Property backpressure : String = "bounded";
}
Attachment RatingEngine.out to UserInterface.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = false;
    Property backpressure : String = "bounded";
}
Attachment UserInterface.out to ScreeningQuestionEngine.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = false;
    Property backpressure : String = "bounded";
}
Attachment ScreeningQuestionEngine.out to ChatGPTIntegration.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment ChatGPTIntegration.out to RatingEngine.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Property resilience_pattern : String = "Retry, CircuitBreaker";
Property delivery : String = "at-least-once";
Property ordering : Boolean = false;
Property authentication : String = "service-to-service";
Property authorization : String = "RBAC";

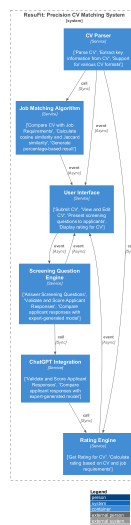
```

}

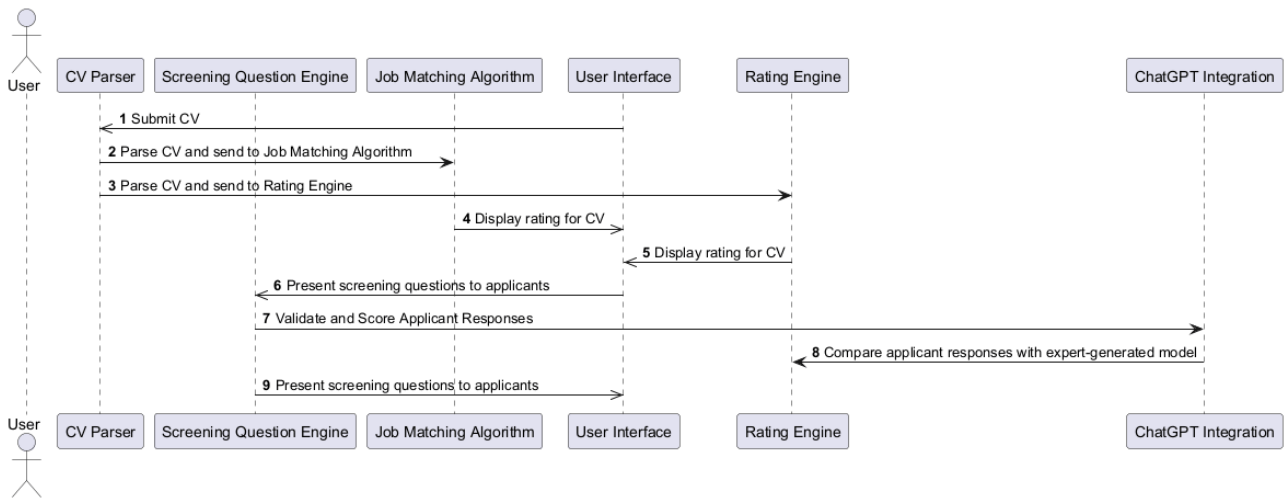
3. Context View



4. Logical View (C4 Container Diagram)



5. Process View (Runtime Interaction)



6. Physical View (Deployment Diagram)

